Código explicado por questão

Proposta 10 — Regularização de sinistros. Cada secção parte do **enunciado**, mostra o **raciocínio**, o **código** que responde (do relatório e do `funcoes_auxiliares.py`) e a **explicação**, com o resultado real. Para cada tarefa há ainda uma frase para dizer na apresentação.

As funções `normalizar_ramo`, `limite_regularizacao` e `cumpre_prazo` e os dicionários `MAPEAMENTO_RAMO` e `LIMITES` estão no `funcoes_auxiliares.py`; o resto está no ficheiro principal, que começa por `from funcoes_auxiliares import normalizar_ramo, limite_regularizacao, cumpre_prazo`.

Secção	Tema	Onde está o código
Tarefa 1	Leitura e qualidade dos dados	principal
Tarefa 2	Coluna valor_pendente	principal
Tarefa 3	limite_regularizacao, cumpre_prazo, fora_prazo	auxiliar + principal
Tarefa 4	Resumo por ramo (groupby)	principal
Tarefa 5	Classe Sinistro	principal
Tarefa 6	Exportação dos prioritários	principal
Questões 1 a 3	Análise e interpretação	código de apoio + resultados

Tarefa 1 — Leitura e verificação de qualidade

Enunciado: Leia os dados e efetue uma verificação de qualidade: tipos, valores em falta e duplicados.

Raciocínio

Antes de analisar, é preciso saber se os dados são fiáveis: *as colunas têm o tipo certo? há valores em falta? há linhas repetidas? as categorias estão bem escritas?* O enunciado exige também ler o ficheiro com `Path` e `pd.read_csv()`.

Código

```
base_dir = Path(__file__).resolve().parent
csv_file = base_dir / "dados_trabalho_10_seguros.csv"

if not csv_file.exists():
    raise FileNotFoundError(
        f"Ficheiro de dados não encontrado: {csv_file}\n"
        "Verifique se o CSV está na mesma pasta do script."
    )

df = pd.read_csv(csv_file)
df[COL_RAMO] = df[COL_RAMO].map(normalizar_ramo)
df[COL_DIAS] = pd.to_numeric(df[COL_DIAS], errors="coerce")
df[COL_RECLAMADO] = pd.to_numeric(df[COL_RECLAMADO], errors="coerce").fillna(0)
df[COL_REGULARIZADO] = pd.to_numeric(df[COL_REGULARIZADO], errors="coerce").fillna(0)

print(df.dtypes)
print(df.isna().sum())
print("Duplicados:", df.duplicated().sum())
print(df[COL_ESTADO].unique(), df[COL_RAMO].unique())
```

Explicação

- `Path(__file__).resolve().parent`: pasta do script. O operador `/` junta o nome do CSV. Assim o ficheiro é encontrado mesmo que o programa corra de outra pasta.
- `csv_file.exists()` + `raise FileNotFoundError`: se faltar o CSV, o programa pára com uma mensagem clara. A parte `f"..."` é uma f-string: `{csv_file}` é substituído pelo caminho.

- `pd.read_csv(csv_file)`: lê o ficheiro para o DataFrame `df` (12 linhas × 7 colunas).
- `.map(normalizar_ramo)`: aplica a função a cada valor da coluna ramo, para uniformizar nomes.
- `pd.to_numeric(..., errors="coerce")`: converte para número; o que não for número vira `NaN` em vez de dar erro. `.fillna(0)` põe 0 nos montantes em falta.
- `dtypes` (tipos), `isna().sum()` (em falta por coluna), `duplicated().sum()` (linhas repetidas) e `unique()` (valores distintos) são a verificação de qualidade pedida.

Resultado

```
tipos:    sinistro, ramo, canal, estado = texto | valores e dias = inteiros (int64)
em falta: 0 em todas as colunas          duplicados: 0
estado:   ['Fechado', 'Pendente']
ramo:     ['Automovel', 'Habitacao', 'Viagem', 'ResponsabilidadeCivil']
```

Nota: o `fillna(0)` corre antes do `isna().sum()`, por isso a contagem de falhas nos montantes ficaria sempre a 0. Nos dados atuais não há falhas, mas numa verificação rigorosa imprime-se `isna().sum()` *antes* de limpar.

Para dizer na apresentação: *"Os dados estão limpos: 12 sinistros, sem valores em falta nem duplicados, e com os quatro ramos bem escritos. Mesmo assim o código está preparado para dados menos limpos."*

Tarefa 2 — Coluna valor_pendente

Enunciado: Crie a coluna `valor_pendente`, igual ao valor reclamado quando o estado é Pendente e zero nos restantes casos.

Raciocínio

É uma regra do tipo *"se… então… senão…"* aplicada a toda a coluna: se o estado é Pendente, fica o valor reclamado; caso contrário, 0. No pandas faz-se sem ciclos, com uma **máscara booleana** (uma coluna de verdadeiro/falso).

Código

```
df["valor_pendente"] = df[COL_RECLAMADO].where(
    df[COL_ESTADO].astype(str).str.strip().eq("Pendente"), 0
)
```

Explicação

- `df[COL_ESTADO].astype(str).str.strip()`: a coluna estado como texto, sem espaços nas pontas.
- `.eq("Pendente")`: compara cada linha com `"Pendente"` e dá `True/False` (a máscara).
- `df[COL_RECLAMADO].where(máscara, 0)`: mantém o valor reclamado onde a máscara é `True` e põe 0 onde é `False`.
- O resultado é guardado numa coluna nova com `df["valor_pendente"] = ...`.

Resultado

```
5 pendentes:  S002 6200 | S004 8400 | S007 1200 | S009 5200 | S012 9800
Fechados:     valor_pendente = 0
Total pendente = 30 800
```

Para dizer na apresentação: *"O valor pendente é o valor reclamado dos processos ainda abertos: 30 800 € em 5 sinistros."*

Tarefa 3 — limite_regularizacao, cumpre_prazo e fora_prazo

Enunciado: Defina `limite_regularizacao(ramo)` (25 dias Automovel e Habitacao, 10 Viagem, 35 ResponsabilidadeCivil) e `cumpre_prazo(ramo, dias_resolucao)`; use ambas para criar a coluna lógica `fora_prazo`.

Raciocínio (o pseudocódigo)

```
limite <- limite_regularizacao(ramo)
SE limite ou dias_resolucao em falta: considerar dentro do prazo
cumpre_prazo <- dias_resolucao <= limite
fora_prazo <- NÃO cumpre_prazo
```

Uma função devolve o **limite** do ramo; outra decide se **cumpre o prazo** comparando os dias com esse limite; a coluna `fora_prazo` é o contrário. Há duas decisões de negócio: o limite é **inclusivo** e, **sem informação, não se assinala atraso**.

Código — funcoes_auxiliares.py

```
LIMITES = {"Automovel": 25, "Habitacao": 25, "Viagem": 10, "ResponsabilidadeCivil": 35}

def normalizar_ramo(ramo):
    if pd.isna(ramo):
        return ramo
    valor = str(ramo).strip()
    return MAPEAMENTO_RAMO.get(valor, valor)

def limite_regularizacao(ramo):
    return LIMITES.get(normalizar_ramo(ramo))

def cumpre_prazo(ramo, dias_resolucao):
    limite = limite_regularizacao(ramo)
    if limite is None or pd.isna(dias_resolucao):
        return True
    return dias_resolucao <= limite
```

Código — ficheiro principal

```
df["fora_prazo"] = ~df.apply(
    lambda linha: cumpre_prazo(linha[COL_RAMO], linha[COL_DIAS]), axis=1
)
```

Explicação

- `LIMITES`: dicionário ramo → dias. `normalizar_ramo` limpa o nome antes de procurar, e `.get(valor, valor)` evita erro se o ramo não existir.
- `limite_regularizacao`: normaliza e procura o limite. Com um só argumento, `.get()` devolve `None` quando o ramo é desconhecido.
- `cumpre_prazo`: se não há limite (`is None`) ou os dias estão em falta (`pd.isna`), devolve `True` (sem informação, não se assinala atraso). Senão devolve `dias <= limite`: 25 dias em Habitacao ainda cumpre.
- `df.apply(função, axis=1)`: chama a função uma vez por linha. `lambda linha: ...` é uma função de uma linha. O `~` inverte: **fora de prazo = não cumpre**.

ramo	dias	limite	cumpre_prazo	fora_prazo
Habitacao	25	25	True	False
Habitacao	32	25	False	True
Viagem	14	10	False	True
Ramo desconhecido	99	None	True	False
Viagem	em falta	10	True	False

Resultado: 7 dos 12 sinistros estão fora de prazo (S002, S004, S005, S007, S008, S009, S012). Os outros 5 cumprem.

Para dizer na apresentação: *"A regra central tem dois pressupostos: o limite é inclusivo e, sem informação, não assinalo atraso, para evitar falsos alarmes. Separei as regras num módulo próprio para estarem isoladas e reutilizáveis."*

Tarefa 4 — Resumo por ramo

Enunciado: Produza um resumo por ramo com número de sinistros, valor reclamado, valor regularizado, valor pendente, dias médios de resolução e percentagem de sinistros fora de prazo.

Raciocínio

Queremos **uma linha por ramo** com seis indicadores. É o padrão *dividir–aplicar–combinar*: dividir a tabela em grupos (um por ramo), aplicar uma operação (contar, somar, média) e combinar num resultado. No pandas faz-se com `groupby()`.

Código

```
resumo = df.groupby(COL_RAMO).agg(
    n_sinistros=(COL_ID, "count"),
    valor_reclamado=(COL_RECLAMADO, "sum"),
    valor_regularizado=(COL_REGULARIZADO, "sum"),
    valor_pendente=("valor_pendente", "sum"),
    dias_medios=(COL_DIAS, "mean"),
    pct_fora_prazo=("fora_prazo", lambda s: s.mean() * 100),
).round(2)
print(resumo)
```

Explicação

- `df.groupby(COL_RAMO)`: separa as linhas em 4 grupos.
- `.agg(nome=(coluna, operação))`: **agregação nomeada**. O nome à esquerda é a coluna nova; o par à direita diz de que coluna e que operação (`count`, `sum`, `mean`).
- `pct_fora_prazo`: `True/False` contam como 1/0, por isso a média é a proporção de sinistros fora de prazo; `* 100` converte em percentagem. A `lambda s` recebe a coluna de cada grupo.
- `.round(2)`: duas casas decimais.

Resultado

Ramo	N.º	Reclamado	Regularizado	Pendente	Dias médios	% fora prazo
Automovel	3	11 100	5 400	5 200	29,33	66,67
Habitacao	3	13 800	7 100	6 200	24,00	33,33
ResponsabilidadeCivil	3	25 800	7 000	18 200	46,00	100,00
Viagem	3	2 830	1 500	1 200	10,00	33,33

Para dizer na apresentação: *"Usei um groupby por ramo com agregações nomeadas. A percentagem fora de prazo é a média de uma coluna lógica multiplicada por 100."*

Tarefa 5 — Classe Sinistro

Enunciado: Defina a classe `Sinistro`, com atributos adequados e o método `valor_pendente()`. Demonstre a sua utilização com um sinistro do CSV.

Raciocínio

Uma classe agrupa **dados** (atributos) e **comportamento** (métodos) de uma entidade. Um sinistro tem identificador, ramo, estado e valor reclamado, e sabe calcular o seu valor pendente. É a mesma regra da Tarefa 2, mas aplicada a **um** sinistro.

Código

```
class Sinistro:
    """Representa um sinistro e calcula o respetivo valor pendente."""

    def __init__(self, id_sinistro, ramo, estado, valor_reclamado):
        self.id_sinistro = id_sinistro
        self.ramo = ramo
        self.estado = estado
        self.valor_reclamado = valor_reclamado

    def valor_pendente(self):
        return self.valor_reclamado if self.estado == "Pendente" else 0

# demonstração (primeiro sinistro pendente do CSV)
linha = df[df[COL_ESTADO] == "Pendente"].iloc[0]
sinistro = Sinistro(
    linha[COL_ID], linha[COL_RAMO], linha[COL_ESTADO], linha[COL_RECLAMADO]
)
print(sinistro.id_sinistro, sinistro.valor_pendente())
```

Explicação

- A **classe** é o molde; `sinistro` é uma **instância** (um sinistro concreto).
- `__init__` é o construtor: corre ao criar a instância e guarda os dados como atributos através de `self`.
- `valor_pendente(self)` é um método: acede aos atributos com `self.estado`. O `return ... if ... else ...` é uma expressão condicional.
- `df[df[COL_ESTADO] == "Pendente"].iloc[0]`: filtra os pendentes e escolhe o primeiro (posição 0), como Series. `linha[COL_ID]` extrai cada valor para o construtor.
- `sinistro.id_sinistro` lê um atributo; `sinistro.valor_pendente()` chama o método (com parênteses).

```
S002 6200
```

Nota: se usares `df.iloc[0]` (a primeira linha de todas), apanhas o S001, que está Fechado, e o resultado é `S001 0`. Funciona, mas não mostra o método a fazer algo. Convém usar um pendente.

Para dizer na apresentação: *"A classe tem 4 atributos e o método valor_pendente, que aplica a regra a um sinistro. Criei uma instância com o S002, que devolve 6 200."*

Tarefa 6 — Exportação dos processos prioritários

Enunciado: Exporte para `sinistros_prioritarios.csv` os processos pendentes ou fora de prazo, ordenados por valor pendente e dias de resolução, ambos de forma decrescente.

Raciocínio

Três passos: **filtrar** (pendente *ou* fora de prazo), **ordenar** (valor pendente e depois dias, do maior para o menor) e **exportar** sem índice técnico.

Código

```
prioritarios = df[(df[COL_ESTADO].astype(str).str.strip().eq("Pendente")) | df["fora_prazo"]]
prioritarios = prioritarios.sort_values(
    by=["valor_pendente", COL_DIAS], ascending=[False, False]
)
prioritarios.to_csv(base_dir / "sinistros_prioritarios.csv", index=False)
```

Explicação

- `df[máscara]` filtra as linhas onde a máscara é verdadeira. A máscara junta duas condições com `|` (OU). Os parênteses são obrigatórios e não se usa `or`, que só funciona com valores únicos.
- `.sort_values(by=[...], ascending=[False, False])`: primeiro por valor pendente decrescente; nos empates, por dias de resolução decrescente.
- `.to_csv(..., index=False)`: grava sem a coluna 0,1,2,... (o índice técnico proibido pelo enunciado).

Resultado — 7 sinistros

Sinistro	Ramo	Canal	Pendente	Dias	Estado
S012	ResponsabilidadeCivil	Corretor	9 800	55	Pendente
S004	ResponsabilidadeCivil	Agente	8 400	45	Pendente
S002	Habitacao	Digital	6 200	32	Pendente
S009	Automovel	Corretor	5 200	42	Pendente
S007	Viagem	Digital	1 200	14	Pendente
S008	ResponsabilidadeCivil	Agente	0	38	Fechado
S005	Automovel	Digital	0	28	Fechado

Os dois fechados (S008 e S005) entram por terem sido resolvidos fora de prazo e ficam no fim porque o valor pendente é 0.

Para dizer na apresentação: *"A lista de prioridades junta os 5 pendentes (todos fora de prazo) e 2 fechados resolvidos tarde, ordenada pelo que há mais dinheiro por resolver."*

Questão 1 — Que ramo necessita de maior atenção operacional?

Enunciado: Que ramo parece necessitar de maior atenção operacional? Fundamente a resposta com pelo menos dois indicadores.

Raciocínio

Não existe um indicador único: escolhem-se indicadores de natureza diferente e vê-se se **apontam para o mesmo ramo**. Aqui: frequência de atrasos (`pct_fora_prazo`), gravidade do atraso (dias médios face ao limite) e dinheiro em jogo (`valor_pendente`). Os dois primeiros saem do resumo da Tarefa 4; o excesso de dias e o peso no pendente calculam-se com duas linhas.

Código de apoio

```
from funcoes_auxiliares import LIMITES

resumo["excesso_dias"] = resumo["dias_medios"] - resumo.index.map(LIMITES)
resumo["pct_pendente_total"] = (
    resumo["valor_pendente"] / resumo["valor_pendente"].sum() * 100
)
```

- `resumo.index.map(LIMITES)`: o índice do resumo são os ramos; o `.map` troca cada ramo pelo seu limite. A diferença dá os **dias acima (ou abaixo) do limite**.
- `valor_pendente / soma * 100`: peso de cada ramo no total pendente.

Ramo	% fora prazo	Dias médios	Limite	Excesso (dias)	Pendente	% do pendente
ResponsabilidadeCivil	100,0	46,0	35	+11,0	18 200	59,1
Automovel	66,7	29,3	25	+4,3	5 200	16,9
Habitacao	33,3	24,0	25	-1,0	6 200	20,1
Viagem	33,3	10,0	10	0,0	1 200	3,9

Resposta

ResponsabilidadeCivil. Tem os três indicadores mais desfavoráveis em simultâneo: todos os sinistros fora de prazo (100%), 11 dias acima do limite em média, e 59% do valor pendente (embora represente 48% do valor reclamado). Concentra ainda os dois maiores pendentes (S012 e S004). Automovel é o segundo (2 em 3 fora de prazo; +4,3 dias).

Nota: com 3 sinistros por ramo, cada sinistro pesa 33 pontos percentuais. A conclusão é indicativa, não estatisticamente robusta, e deve ser dita assim. Nos pendentes, os dias são provavelmente tempo decorrido e não duração final.

Questão 2 — Concentração por canal ou por ramo?

Enunciado: Os processos prioritários concentram-se num canal de participação ou num ramo específico?

Raciocínio

Compara-se a **composição dos prioritários** com a **composição de todos os sinistros**. Se um canal pesar muito mais nos prioritários do que na base completa, há concentração. Cada canal tem 4 sinistros e cada ramo tem 3, por isso a base é equilibrada e a comparação é justa.

Código (já no script)

```
print(df[COL_CANAL].value_counts(normalize=True))
print(prioritarios[COL_CANAL].value_counts(normalize=True))
print(pd.crosstab(prioritarios[COL_CANAL], prioritarios[COL_RAMO]))
```

- `.value_counts(normalize=True)`: conta e converte em proporção (soma 1). A 1.ª linha dá a composição da base; a 2.ª, a dos prioritários.
- `pd.crosstab(a, b)`: tabela de contagens cruzando canal (linhas) e ramo (colunas).

Canal	Na base (n=12)	Prioritários (n=7)	Pendente (€)	% fora de prazo
Digital	4 (33,3%)	3 (42,9%)	7 400	75%
Corretor	4 (33,3%)	2 (28,6%)	15 000	50%
Agente	4 (33,3%)	2 (28,6%)	8 400	50%

Ramo	Na base (n=12)	Prioritários (n=7)
ResponsabilidadeCivil	3 (25,0%)	3 (42,9%)
Automovel	3 (25,0%)	2 (28,6%)
Habitacao	3 (25,0%)	1 (14,3%)
Viagem	3 (25,0%)	1 (14,3%)

Resposta

A concentração é **mais clara por ramo**: ResponsabilidadeCivil passa de 25% da base para 43% dos prioritários. Por canal, o Digital sobe de 33% para 43% e tem a maior percentagem de atrasos (75%), mas a

diferença é **um único sinistro**. O Corretor, por sua vez, tem o maior valor pendente (15 000 €, 49% do total), por causa do S012 e do S009. Nenhum canal se destaca de forma consistente nos três critérios, por isso o padrão aponta para o **ramo** e não para o canal.

Nota: a tabela por canal (pendente e % fora de prazo) vem de um `groupby(COL_CANAL)` análogo ao da Tarefa 4; não está no script atual e pode ser acrescentada se a quiseres no relatório.

Questão 3 — Variáveis adicionais para interpretar atrasos

Enunciado: Que variáveis adicionais ajudariam a interpretar atrasos sem atribuir automaticamente responsabilidade a clientes ou equipas?

Raciocínio

Esta questão não precisa de código: pede **pensamento crítico**. Um atraso (dias acima do limite) é um *sintoma*; para o explicar é preciso saber *o que aconteceu durante o processo*. As variáveis devem descrever o contexto e o processo, não culpar pessoas. Isto liga-se ao objetivo do enunciado: apoiar a organização da equipa, não decidir sobre sinistros.

Variáveis sugeridas

Variável	Para que serve
Complexidade do sinistro (peritagem, litígio, n.º de intervenientes)	Atrasos podem refletir casos mais difíceis, não pior desempenho
Datas por etapa (participação, abertura, peritagem, decisão, pagamento)	Mostra em que fase o processo parou
Motivo de pendência (documentos em falta, perícia, terceiros, decisão interna)	Distingue causas externas de internas, sem culpar à partida
Carga de trabalho e capacidade da equipa no período	Separa atraso individual de sobrecarga
Data de referência da análise	Nos pendentes, os dias são tempo decorrido até à data
Tipo de cobertura e valor segurado	Compara processos comparáveis

Como entrariam no código: com datas por etapa, os dias de cada fase calculavam-se com `pd.to_datetime()` e uma subtração (exemplo ilustrativo; estas colunas **não existem** no CSV atual):

```
df["dias_ate_peritagem"] = (
    pd.to_datetime(df["data_peritagem"]) - pd.to_datetime(df["data_participacao"])
).dt.days
df.groupby("ramo")["dias_ate_peritagem"].mean()
```

Para dizer na apresentação: *"Um atraso mostra que algo demorou, mas não porquê. Precisaria de datas por etapa, do motivo da pendência e da complexidade do caso, para analisar processos e não pessoas."*

Conclusão e recomendação final

Conclusão: 7 dos 12 sinistros (58,3%) estão fora de prazo e 30 800 € continuam pendentes. A ResponsabilidadeCivil concentra o maior problema; por canal, a evidência é fraca com tão poucos dados.

Recomendação: iniciar o acompanhamento operacional pela ResponsabilidadeCivil, em particular os sinistros S012 e S004 (18 200 € em conjunto), seguindo-se S009 (Automovel) e S002 (Habitacao); recolher as variáveis da Questão 3 e repetir a análise com mais dados antes de concluir sobre canais. A lista é para organizar o trabalho da equipa, não para decidir sobre sinistros individuais.

Limitações a referir: amostra muito pequena (12 sinistros); nos pendentes, `dias_resolucao` é provavelmente tempo decorrido; limites fixos por ramo, sem ter em conta a complexidade; dados fictícios e sem datas.